

Tema lietuvių kalba:

Asmeninių finansų sekimo internetinė informacinė sistema

Tema anglų kalba:

Personal finance tracker web app

Projekto idėjos aprašymas:

Sukurta naršyklėje veikianti asmeninio biudžeto stebėjimo internetinė informacinė sistema. Sistema leidžia įvesti finansines operacijas, jas kategorizuoti, peržiūrėti mėnesinį biudžeto balansą ir statistinę informaciją.

Atliko: Lukas Šerelis

Grupė: II

Vadovas: dr. Andrius Misiukas Misiūnas

Pagrindinės technologijos

- Next.js and React: leidžia daryti Server-Side Rendering greitam krovimui ir turi puikų routingą.
- TypeScript: tipų saugumas padeda išvengti klaidų rašant kodą.
- Tailwind CSS: greitas UI konstravimas be didelio vargo su atskirais CSS failais.
- SQLite: lengva, nereikia atskiro serverio, viskas saugoma faile.
- Node.js: užtikrina sklandų API komunikaciją tarp front-end ir DB.

Planuojamas funkcionalumas

I iteracija

- Vartotojų CRUD ir autentifikacija (Login/Register).
 - Vartotojų patvirtinimo sistema: naujai užsiregistravę vartotojai gauna "Pending" statusą ir negali naudotis sistema, kol administratorius jų nepatvirtina.
 - Administratoriaus panelė: galimybė administratoriui matyti visus vartotojus, juos redaguoti, trinti arba keisti jų statusą (pirmas užsiregistravęs vartotojas automatiškai tampa admin).
- Mėnesio sheets kūrimas.
- Pajamų/Išlaidų kategorijų CRUD: galimybė susikurti savo pajamas ir išlaidas (pvz. "maistas" ar "alga"), nes be kategorijų nebus kur vesti duomenų.

II iteracija

- Operacijų (Pajamų/Išlaidų) CRUD: galimybė įvesti ar redaguoti sumas, priskirti datas ir kategorijas.
- Finansinės būsenos suvestinė (Dashboard): pagrindinis ekranas, rodantis tam tikras statistikas ir palyginimus paskutinių mėnesių ar dienų.
- Dark ir White Theme.

III iteracija

- **Mėnesio Dashboard:** sistema analizuoja praėjusį mėnesį ir pateikia įžvalgas (pvz. "šį mėnesį maistui išleisdote 20% daugiau nei vidutiniškai").
 - Rodo dabartinį balansą (pliusas/minusas) ir pajamas/išlaidas pagal kategorijas.
 - Turto (Capital) sekimas: skirstymas į Savings, Cash ir t.t. su procentiniu pasiskirstymu.
- **Profilio dalinimasis:** galimybė suteikti prieigą kitam registruotam vartotojui peržiūrėti tavo mėnesio ataskaitas.

IV iteracija

- Pasikartojančios operacijos: galimybė nustatyti, kad "Nuoma" ar "Alga" įsirašytų automatiškai kas mėnesį.
- Išlaidų skaidymas: galimybė vienkartinės metinės prenumeratas ar stambius pirkinius tolygiai paskirstyti per pasirinktą mėnesių skaičių. Tai leidžia vartotojui matyti realų mėnesio biudžeto apkrovimą, o ne vienkartinį "duobėtą" balansą.
- Adaptyvus dizainas: optimizacija, kad viskas atrodytų gražiai tiek ant kompiuterio tiek ant telefono.

Įgyvendintas funkcionalumas

I iteracija

- Vartotojų CRUD ir autentifikacija (Login/Register).
 - Vartotojų patvirtinimo sistema: naujai užsiregistravę vartotojai gauna "Pending" statusą ir negali naudotis sistema, kol administratorius jų nepatvirtina.
 - Administratoriaus panelė: galimybė administratoriui matyti visus vartotojus, juos redaguoti, trinti arba keisti jų statusą (pirmas užsiregistravęs vartotojas automatiškai tampa admin).
- Mėnesio sheets kūrimas.
- Pajamų/Išlaidų kategorijų CRUD: galimybė susikurti savo pajamas ir išlaidas (pvz. "maistas" ar "alga"), nes be kategorijų nebus kur vesti duomenų.

II iteracija

- Operacijų (Pajamų/Išlaidų) CRUD: galimybė įvesti ar redaguoti sumas, priskirti datas ir kategorijas.
- Finansinės būsenos suvestinė (Dashboard): pagrindinis ekranas, rodantis tam tikras statistikas ir palyginimus paskutinių mėnesių ar dienų.
- Dark ir White Theme.

III iteracija

- **Capital Tab:**
 - Turto (Capital) sekimas: skirstymas į Savings, Cash ir t.t. su procentiniu pasiskirstymu.
- **Profilio dalinimasis:** galimybė suteikti prieigą kitam registruotam vartotojui peržiūrėti tavo mėnesio ataskaitas.

IV iteracija

- Pasikartojančios operacijos: galimybė nustatyti, kad "Nuoma" ar "Alga" įsirašytų automatiškai kas mėnesį.
- Išlaidų skaidymas: galimybė vienkartinės metinės prenumeratas ar stambius pirkinius tolygiai paskirstyti per pasirinktą mėnesių skaičių. Tai leidžia vartotojui matyti realų mėnesio biudžeto apkrovimą, o ne vienkartinį "duobėtą" balansą.
- Adaptyvus dizainas: optimizacija, kad viskas atrodytų gražiai tiek ant kompiuterio tiek ant telefono.

Nefunkciniai reikalavimai

- Prieinamumas: sistema turi palaikyti Dark/White režimus ne tik dėl estetikos, bet ir dėl vartotojų, turinčių regos sutrikimų.
- Privatumas: po pasidalinimo profiliu, vartotojas turi turėti galimybę bet kada atšaukti prieigą kitam vartotojui.
- Našumas: net ir turint daug operacijų, balanso skaičiavimas ir apžvalga turi būti generuojami greičiau nei per 1 sekundę.

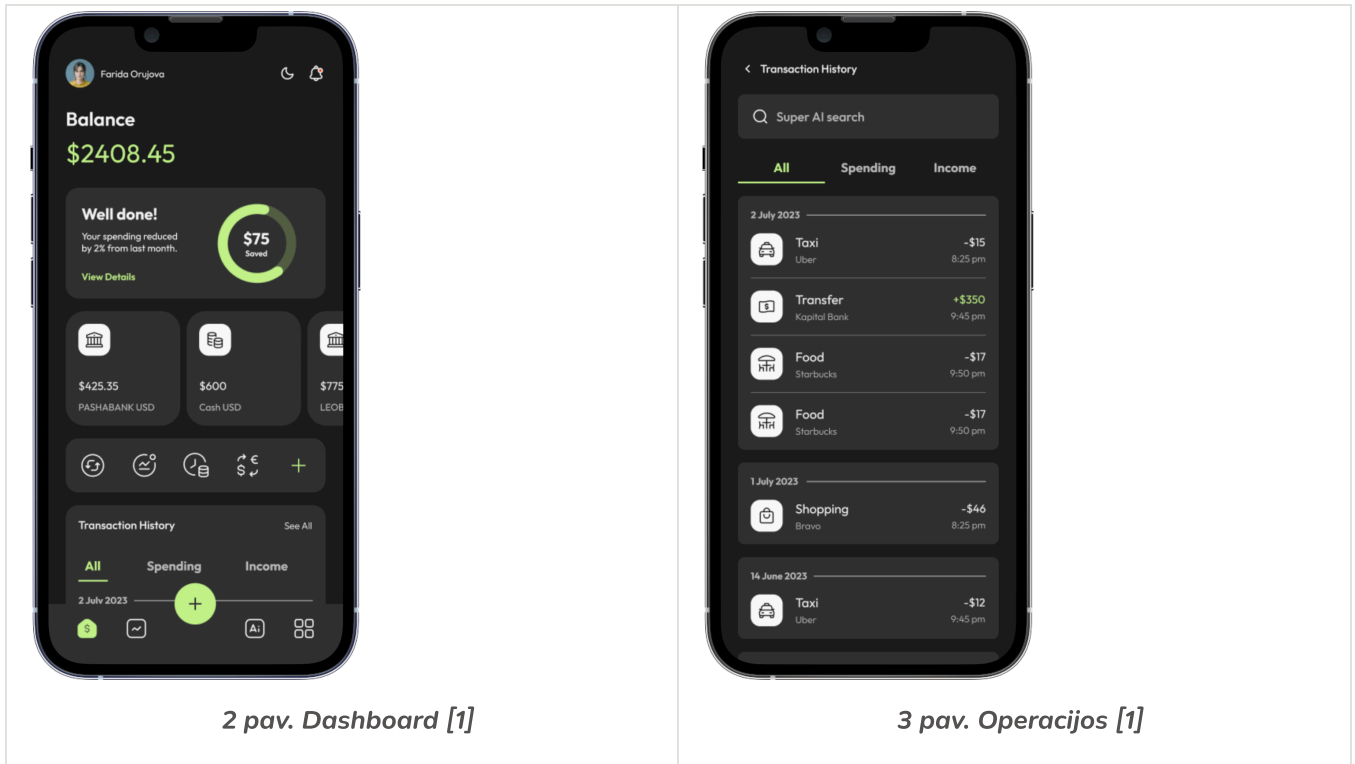
Inspiracija

	A	B	C	D	E	F	G	H	I	J
1	Category	Income	Category	Expense						
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										
21										
22										
23										
24										
25										
26										
27										
28										
29										
30										
31										
32										
33										
34										
35										
36										

1 pav. Vieno mėnesio apžvalga

Vizija

Mobile



2 pav. Dashboard [1]

3 pav. Operacijos [1]

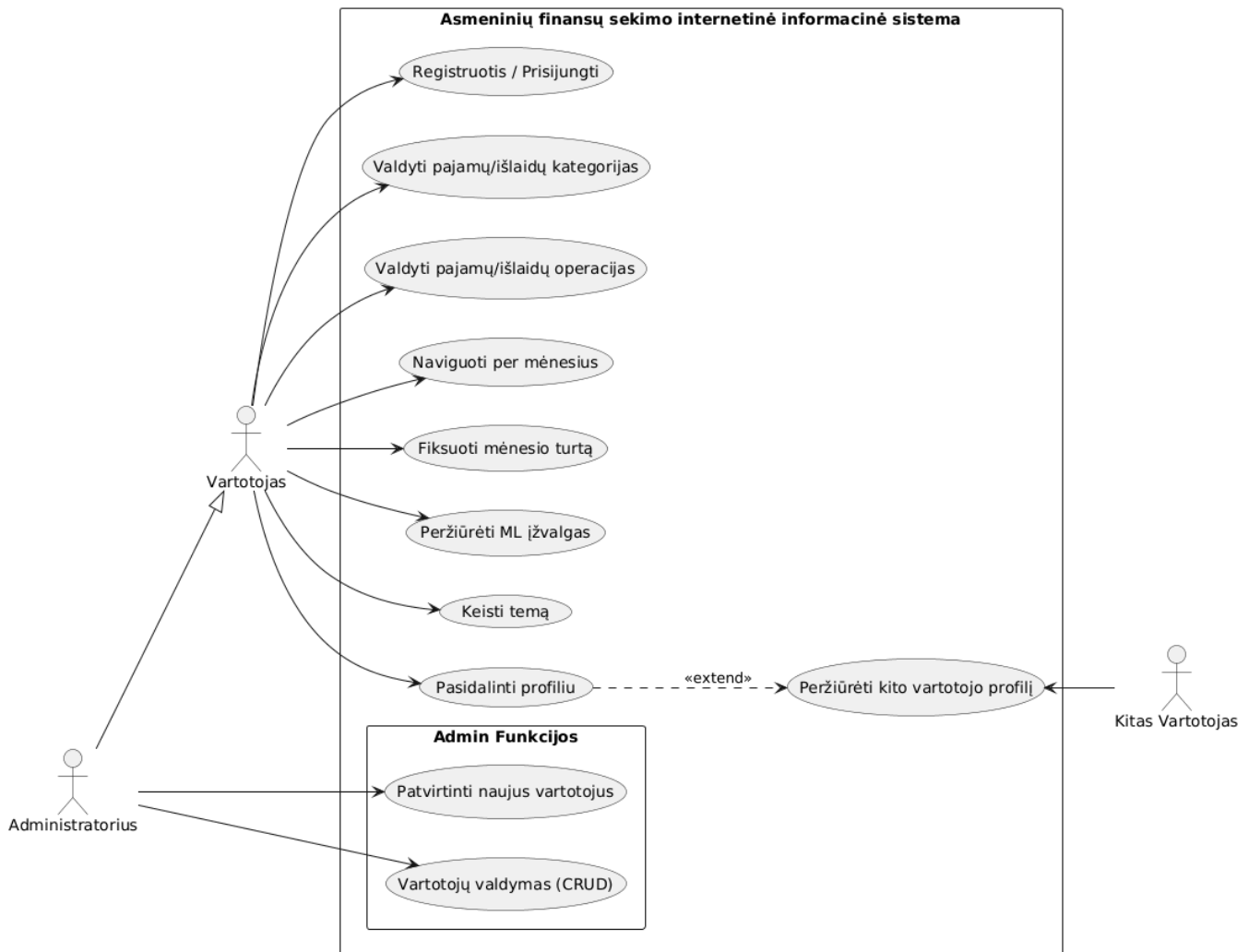
Computer



4 pav. Dashboard [2]

Use-case UML diagrama

Originali



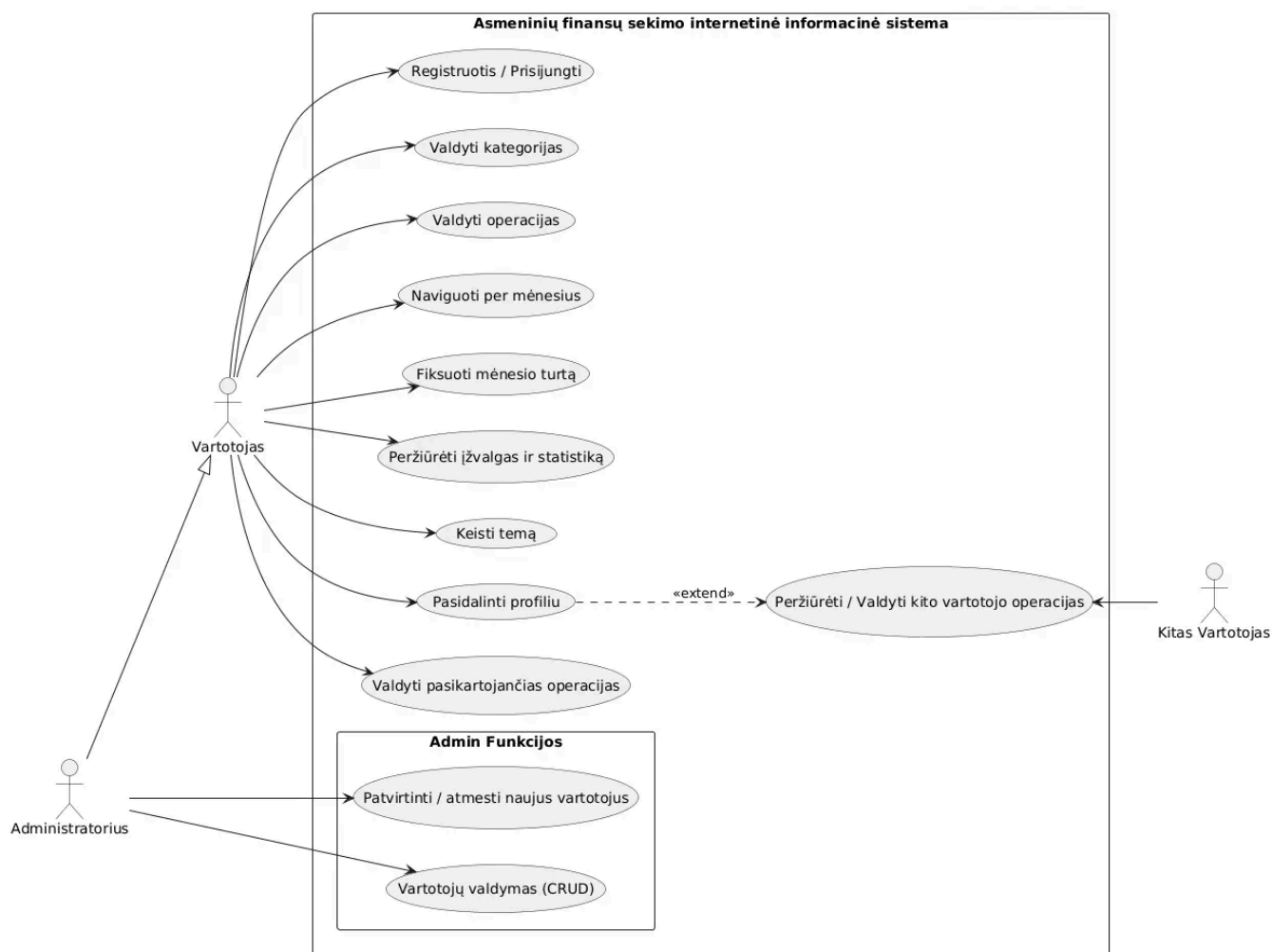
5 pav. Originali use-case UML diagrama

Sistema skirstoma į tris tipus vartotojų: pagrindinį vartotoją, administratorių bei kitą registruotą vartotoją, kuriam gali būti suteikta prieiga prie kito profilio peržiūros.

Administratorius šioje architektūroje paveldi visas paprasto vartotojo teises, tačiau papildomai turi išskirtines teises valdyti naudotojų sąrašą bei patvirtinti naujas registracijas.

Vartotojas gali registruotis, prisijungti, valdyti kategorijas ir operacijas, peržiūrėti dabartinį ir senesnius mėnesius, įvesti ir sekti savo turtą, peržiūrėti mėnesio įžvalgas, keisti temą, pasidalinti profiliu ir peržiūrėti kito vartotojo profilį.

Atnaujinta

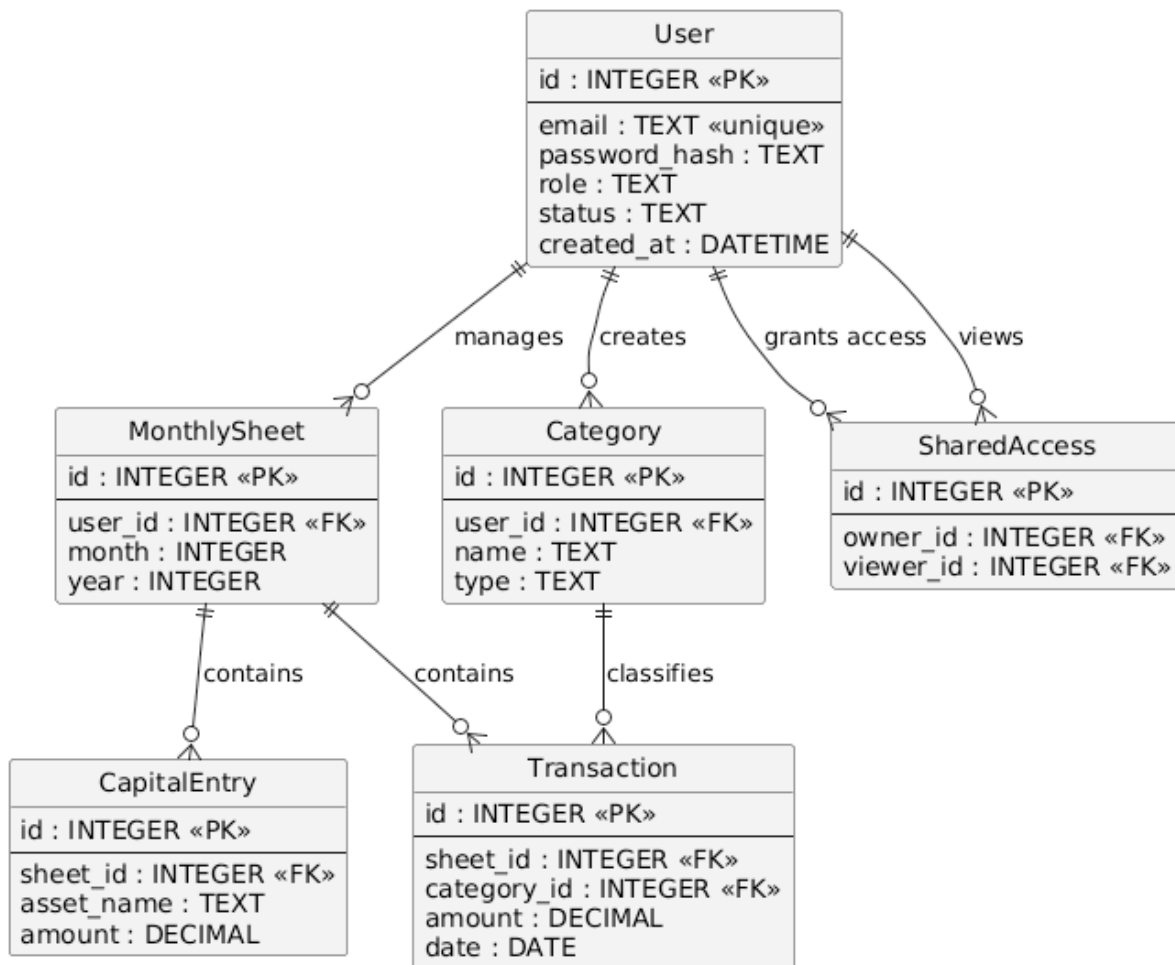


6 pav. Atnaujinta use-case UML diagrama

- Pridėta - "Valdyti pasikartojančias operacijas" (atskiras puslapis)
- Kitas vartotojas gali ne tik peržiūrėti bet ir valdyti kito vartotojo operacijas.

Scheme of database

Originali



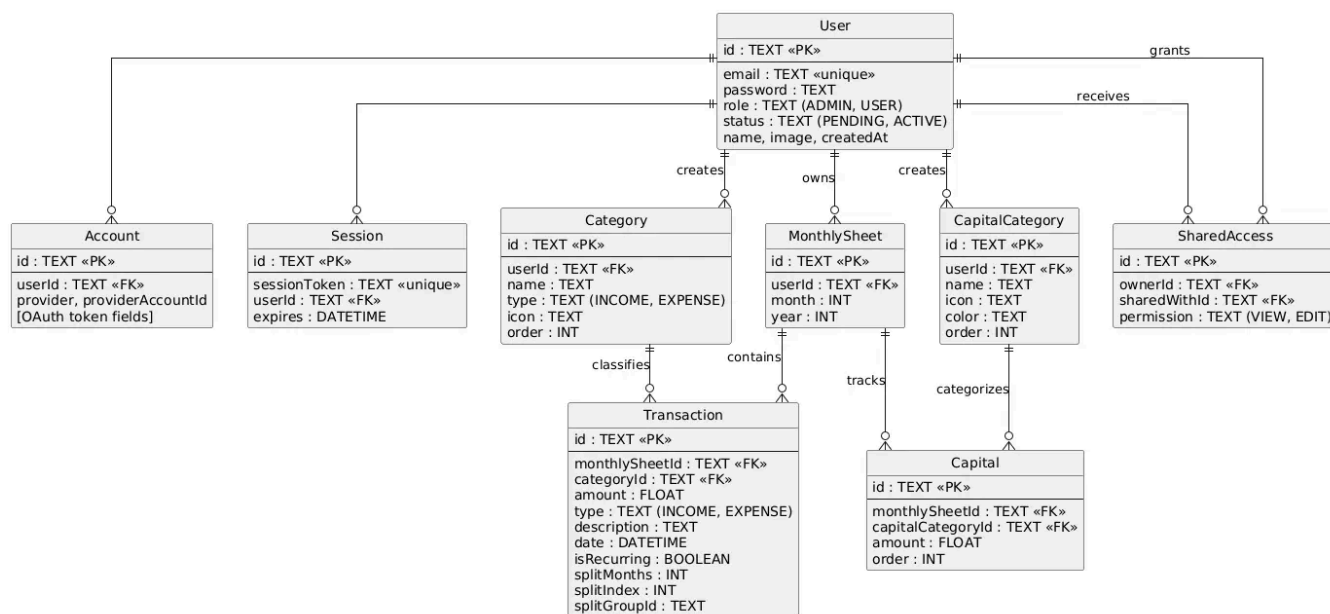
7 pav. Originali duomenų bazės schema

Siekiant užtikrinti pajamų ir išlaidų atskyrimą Category lentelė turi type atributą. Tai leidžia vartotojo sąsajai filtruoti kategorijas: pildant išlaidų operaciją, vartotojui pateikiamos tik išlaidų tipo kategorijos ir atvirkščiai.

Lentelė MonthlySheet yra pagrindinis mėnesio sesijos identifikatorius, prie kurio yra jungiami visi to laikotarpio finansiniai įvykiai ir turto operacijos.

Ryšiai tarp lentelių nustatyti naudojant išorinius raktus, o vartotojų teisių valdymas ir profilių dalinimasis realizuojamas per User ir SharedAccess lentelių sąveiką.

Atnaujinta

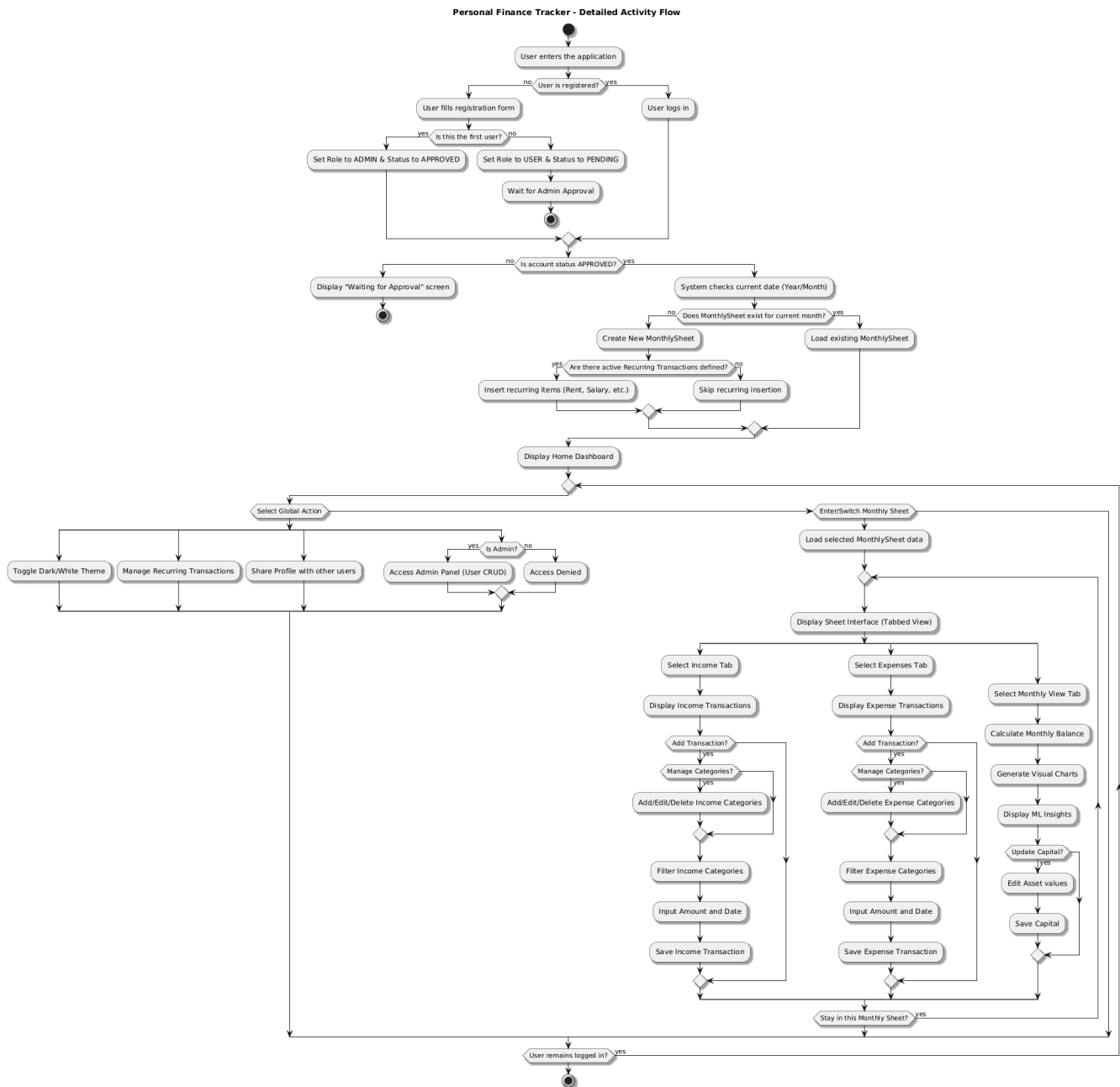


8 pav. Atnaujinta duomenų bazės schema

- Originalaus plano schema turėjo 6 lenteles. Galutinė realizacija išsiplėtė iki 9 lentelių - pridėti NextAuth reikalingi modeliai (Account, Session).
- Capital susietas su CapitalCategory vietoje laisvojo teksto assetName.
- Išplėstas SharedAccess su permission lauku.
- Transaction modelis papildytas split laukais.
- Capital taip pat turi categories kaip ir transactions.

Activity UML diagrama

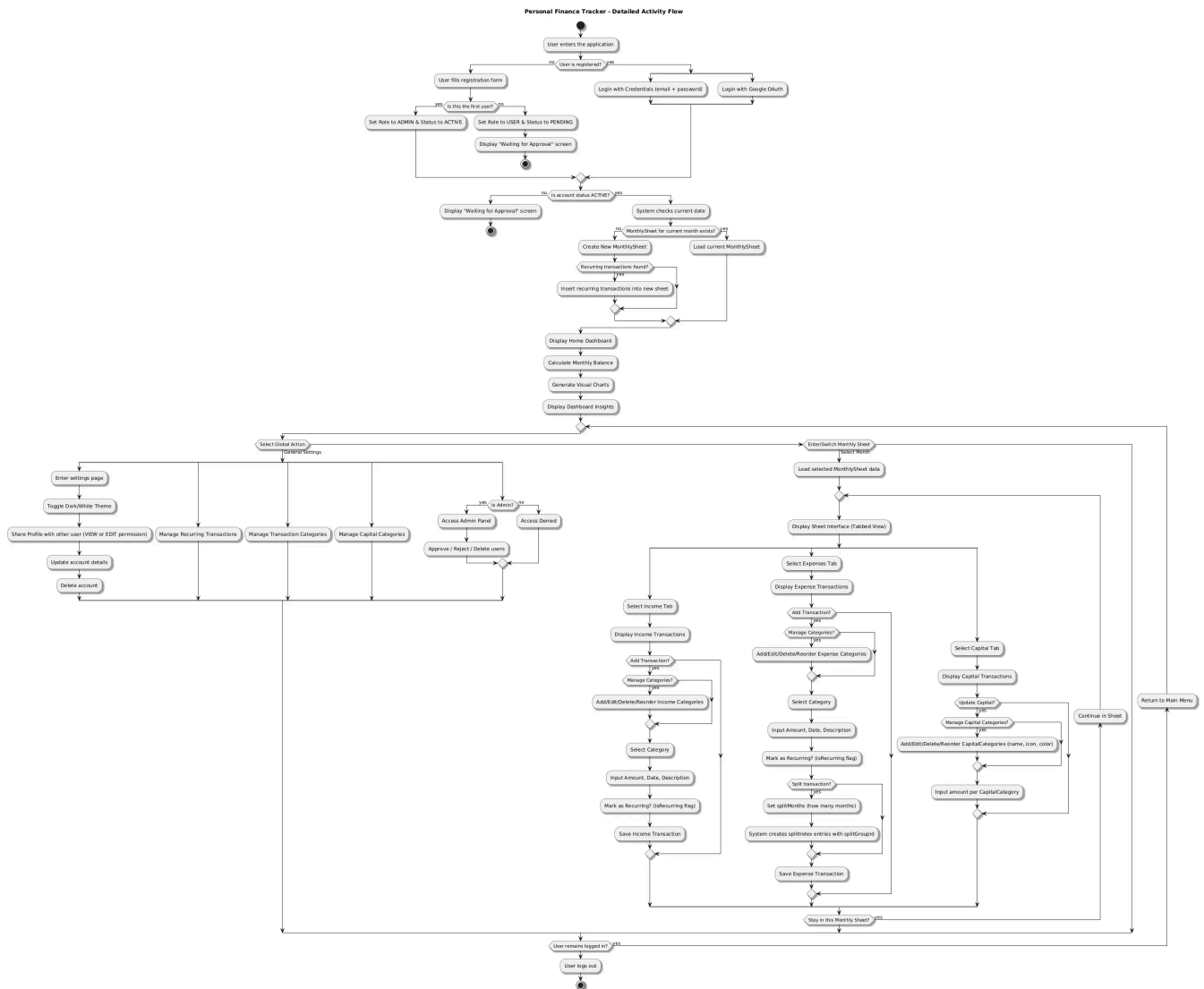
Originali



9 pav. Originali activity UML diagrama

Sistema suprojektuota taip, kad pirmasis užsiregistravęs asmuo automatiškai įgyja administratoriaus teises, o vėlesni vartotojai privalo laukti jo patvirtinimo. Sėkmingai prisijungus, sistema atlieka automatinį mėnesio sheet inicijavimą: patikrina einamąją datą, sukuria naują darbo erdvę ir įrašo pasikartojančias operacijas.

Sistemos navigacija pavaizduota naudojant split blokus, kurie geriausiai atspindi tabs principu veikiančių vartotojo sąsają. Vartotojas gali laisvai rinktis tarp globalių nustatymų (temos keitimas, profilio dalinimasis) arba konkretaus mėnesio sheet valdymo. Sheet'o viduje logika skirstoma į tris izoliuotus modulius: pajamos, išlaidos ir mėnesio apžvalga.

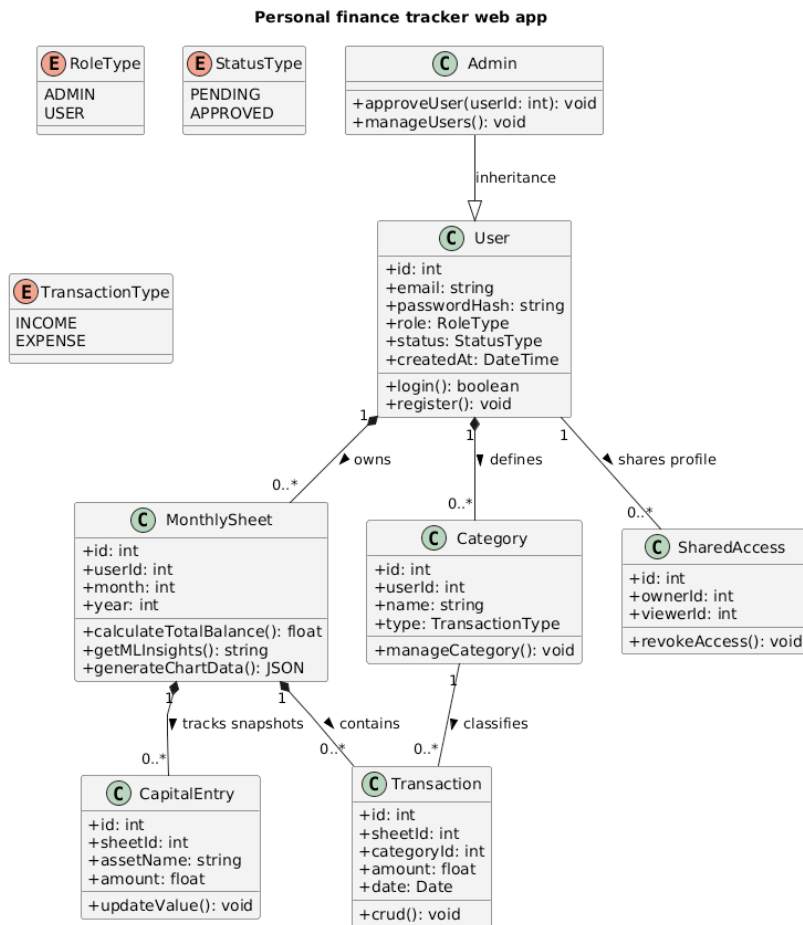


10 pav. Atnaujinta activity UML diagrama

- Statuso pavadinimas - originale APPROVED, realizacijoje ACTIVE
- Prisijungimas - pridėtas Google OAuth
- Nebėra planuoto atskirto Monthly view tab kuriame būtų įvairi statistika - jis pakeistas capital tab'u
- Home Dashboard'e yra Monthly view tab'e planuotas funkcionalumas
- Transaction įvedimas - realizacijoje galima pažymėti isRecurring ir nustatyti split per kelis mėnesius
- CapitalCategory su spalva/ikona/rikiavimas

UML class diagrama

Originali

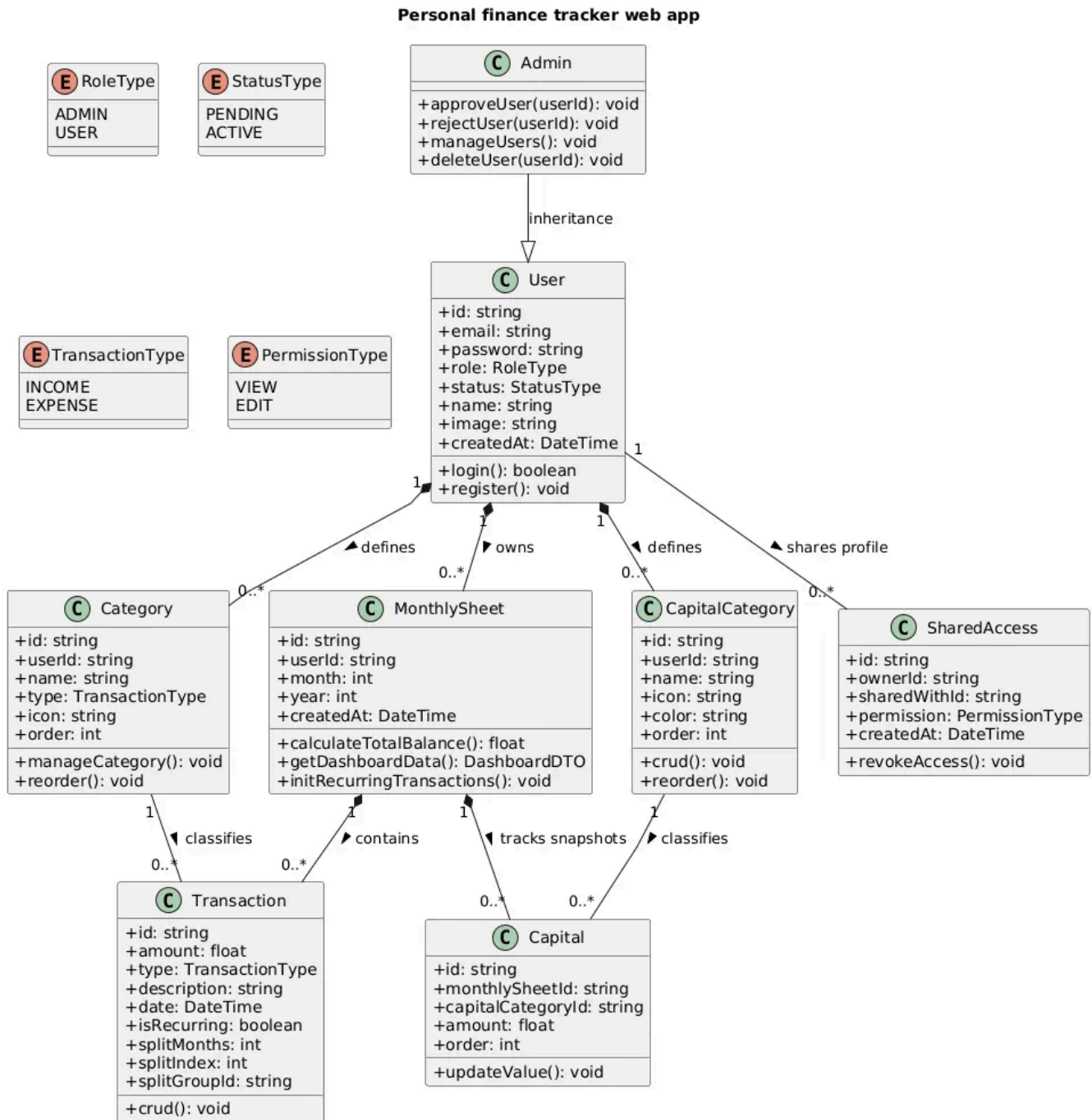


11 pav. Originali UML klasių diagrama

MonthlySheet klasė ne tik saugo ryšius su operacijomis ir turto fiksavimu, bet ir turi loginius metodus (calculateTotalBalance, getMLInsights), kurie atsakingi už dinaminį duomenų apdorojimą ir analitikos generavimą vartotojui.

Ryšiai tarp klasių nurodo griežtą priklausomybę, pavyzdžiui, operacijos ir turto įrašai negali egzistuoti be konkretaus mėnesio lakšto, o kategorijų filtravimas pagal tipą (INCOME, EXPENSE) užtikrinamas per TransactionType.

Atnaujinta



12 pav. Atnaujinta UML klasių diagrama

- Pridėta CapitalCategory klasė
- Transaction pridėta split laukai (splitMonths, splitIndex, splitGroupId) ir isRecurring
- SharedAccess papildyta permission atributu
- MonthlySheet metodas getMLInsights() pervadintas į getDashboardData() - realizuota kaip statistikos agregavimas, ne ML
- Admin klasėje pridėti deleteUser() ir rejectUser() metodai

Projekto repozitorija

Projekto kodas: <https://github.com/lukrencijus/finance-web-app>

Projekto struktūra

Architektūriniai sprendimai

- Next.js App Router leidžia kiekvienam puslapiui turėti tiek server, tiek client komponentus. Projektas naudoja šį principą:
 - page.tsx failai yra Server Components - duomenys gaunami tiesiogiai iš DB per Prisma, be papildomo API sluoksnio.
 - client.tsx failai yra Client Components - interaktyvios UI dalys su useState/useRouter.
 - actions.ts failai kiekvienam moduliui - Next.js Server Actions, kurie atstoja tradicinį REST API.

Autentifikacija

- Autentifikacija realizuota naudojant NextAuth.js. Palaikomi du prisijungimo būdai:
 - Credentials - el. paštas + slaptažodis (bcryptjs maiša)
 - Google OAuth 2.0 - prisijungimas per Google paskyrą
- Next.js middleware tikrina kiekvieno užklauso autentifikacijos būseną ir nukreipia:
 - Neprisijungusius → /sign-in
 - PENDING statusu vartotojus → /pending
 - Prisijungusius, bandančius pasiekti /sign-in → / (pagrindinis puslapis)

Duomenų bazė ir Object-Relational Mapper

Duomenų bazė - SQLite (failas dev.db), valdoma per Prisma ORM su @prisma/adapters-better-sqlite3 adapteriu. Leidžia dirbti su duomenų baze naudojant normalų TypeScript kodą, vietoje SQL užklausų.

Modelis	Paskirtis
User	Vartotojas: el. paštas, slaptažodis, role (USER/ADMIN), status (PENDING/ACTIVE)
Account	OAuth paskyros ryšys
Session	Sesijų valdymas
Category	Pajamų/išlaidų kategorija su tipo atributu ir ikona
MonthlySheet	Mėnesio sesija: month + year + userId
Transaction	Finansinė operacija: suma, tipas, data, kategorija, sheet'as, split laukai
Capital	Turto įrašas konkrečiam sheet'ui ir CapitalCategory
CapitalCategory	Vartotojo apibrėžta turto kategorija su spalva
SharedAccess	Profilio dalinimasis: owner + sharedWith + permission

Pagrindinės bibliotekos

Biblioteka	Paskirtis
next-auth	Autentifikacija
prisma + better-sqlite	ORM ir SQLite adapteris
next-themes	Dark/White temos perjungimas
shadcn/ui + radix-ui	UI komponentų biblioteka
tailwindcss	Stiliai
@dnd-kit/core + sortable	Drag and drop kategorijų rikiavimui
zod v4	Formos duomenų validacija
bcryptjs	Slaptažodžių maiša
lucide-react	Ikonų biblioteka
chart.js	Grafikai dashboard'e

Pakeitimai nuo pradinio plano

- Pradiniame plane profilio dalinimasis buvo planuojamas tik su peržiūros teise. Realizacijoje pridėtas permission atributas (VIEW arba EDIT), leidžiantis suteikti ir redagavimo teises.
- Prie Category ir CapitalCategory modelių pridėtas order laukas, o UI naudoja @dnd-kit biblioteką drag-and-drop funkcionalumui - vartotojas gali rankiniu būdu keisti kategorijų eiliškumą.
- Use-case diagramoje ir pradiniame plane buvo numatytas getMLInsights() metodas. Realizacijoje šis funkcionalumas realizuotas kaip statistika iš DB - lyginamos mėnesių sumos ir skaičiuojami procentiniai pokyčiai, nenaudojant ML modelio.
- Prisijungimas - pridėtas Google OAuth.

Paleidimo instrukcija

Klonavimas

```
git clone https://github.com/lukrencijus/finance-web-app.git
cd finance-web-app
```

Įdiegimas

```
npm install
```

Aplinkos kintamieji

- Nukopijuokite .env.example į .env ir užpildykite reikiamas reikšmes:

```
cp .env.example .env
```

- AUTH_SECRET:

```
npx auth secret
```

- Google OAuth konfigūracija (AUTH_GOOGLE_ID, AUTH_GOOGLE_SECRET) yra neprivaloma - jei nereikia Google prisijungimo, šiuos laukus galima palikti tuščius.

Duomenų bazės paruošimas

Sukurti SQLite duomenų bazę ir pritaikyti visas migracijas:

```
npx prisma migrate dev
```

Kūrimo serverio paleidimas

```
npm run dev
```

Pirmo vartotojo kūrimas

Pirmasis užsiregistravęs vartotojas automatiškai gauna ADMIN teises ir ACTIVE statusą. Visi kiti vartotojai gaus PENDING statusą ir turės laukti administratoriaus patvirtinimo. Aplikacija bus prieinama adresu: <http://localhost:3000>

Dirbtinio intelekto naudojimas

- Projekto kūrimo metu dirbtinis intelektas buvo naudojamas:
 - UI komponentų stilizavimui - mygtukų, kortelių, išdėstymo Tailwind CSS klasių generavimui
 - Pasikartojančių šablonų spartinimui, pvz. panašios struktūros komponentai
- Savarankiškai suprojektuota ir realizuota:
 - Visa verslo logika - mėnesio sheet'ų kūrimas, pasikartojančių operacijų mechanizmas, išlaidų skaidymas per mėnesius
 - Autentifikacija - NextAuth.js konfigūracija su Credentials ir Google OAuth, administratoriaus bootstrap logika
 - Middleware - apsauga ir redirect'ai pagal vartotojo statusą
 - Visos CRUD operacijos per Next.js Server Actions
 - Duomenų bazės schema ir migracijų istorija (Prisma)
 - Dashboard duomenų agregavimas ir statistikos skaičiavimas

Literatūros ir šaltinių sąrašas

[1] [Fintrack app — AI-Powered Personal Finance app](#)

[2] [Financial Tracker Dashboard Redesign](#)